

CO3 - AT1 - Analytical Problem solving

[← Back](#)

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks

Q2
CO3 - AT1 - Analytical Problem Solving
20 marks

Q3
CO3 - AT1 - Analytical Problem Solving
20 marks

Q4
CO3 - AT1 - Analytical Problem Solving
20 marks

Q5
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q5 CO3 - AT1 - Analytical Problem Solving

20 marks

5. Accept a string containing repeated characters. Compress it by replacing consecutive repeated characters with the character followed by its count.

Example:

Input:

aaabbbccccdd

Output:

a3b3c4d2

Also calculate the compression ratio.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 int main() {
4     char str[200];
5     char comp[400];
6     scanf("%s", str);
7     int len = (int)strlen(str);
8     int i = 0, j = 0;
9     while (i < len) {
10         char ch = str[i];
11         int count = 1;
12         while (i + 1 < len && str[i] == str[i + 1]) {
13             count++;
14             i++;
15         }
16         comp[j++] = ch;
17         char cntStr[20];
18         int k = 0;
19         while (count > 0) {
20             cntStr[k++] = (count % 10) + '0';
21             count /= 10;
22         }
23         for (int t = k - 1; t >= 0; t--) {
24             comp[j++] = cntStr[t];
25         }
26         i++;
27     }
```

Input

aaabbbccccdd

Output

CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks

Q2
CO3 - AT1 - Analytical Problem Solving
20 marks

Q3
CO3 - AT1 - Analytical Problem Solving
20 marks

Q4
CO3 - AT1 - Analytical Problem Solving
20 marks

Q5
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q4 CO3 - AT1 - Analytical Problem Solving

20 marks

4. Given N strings, write a C program to:
- Sort them lexicographically.
 - Find the longest and shortest strings.
 - Count duplicate strings.
 - Display strings beginning with vowels.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 int main() {
5     int N,i,j,max=0,min=0,dup=0;
6     char a[100][200], t[200];
7     printf("Enter number of strings (N): ");
8     scanf("%d",&N);
9     for(i=0;i<N;i++) scanf("%199s",a[i]);
10    for(i=0;i<N;i++){ // longest/shortest indices (by length)
11        int L=strlen(a[i]);
12        if(L>strlen(a[max])) max=i;
13        if(L<strlen(a[min])) min=i;
14    }
15    for(i=0;i<N-1;i++) // lexicographic sort
16        for(j=i+1;j<N;j++)
17            if(strcmp(a[i],a[j])>0){ strcpy(t,a[i]); strcpy(a[i],a[j]);
18                strcpy(t,a[j]);
19            }
20    for(i=1;i<N;i++) // duplicate count (extra occurrences)
21        if(!strcmp(a[i],a[i-1])) dup++;
22    printf("\nSorted:\n");
23    for(i=0;i<N;i++) printf("%s\n",a[i]);
24
25    printf("\nLongest: %s\nShortest: %s\nDuplicates(extra occurrences): %d",a[max],a[min],dup);
26
27    printf("\nVowel starting strings:\n");
28    for(i=0;i<N;i++){
29        char c=tolower(a[i][0]);
30        if(c=='a' || c=='e' || c=='i' || c=='o' || c=='u') printf("%s\n",a[i]);
31    }
```

Input

7
apple
mango
orange

Output

Active
Go to S

CO3 - AT1 - Analytical Problem solving

[← Back](#)

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks

Q2
CO3 - AT1 - Analytical Problem Solving
20 marks

Q3
CO3 - AT1 - Analytical Problem Solving
20 marks

Q4
CO3 - AT1 - Analytical Problem Solving
20 marks

Q5
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q2 CO3 - AT1 - Analytical Problem Solving

20 marks

2. Given an array of integers, write a C program that:

- Determines whether the array is sorted.
- If sorted, performs Binary Search.
- Otherwise, performs Linear Search.
- Displays the search method used and the number of comparisons performed.

Your Code

```
1 #include<stdio.h>
2 int main() {
3     int a[10], n, i, key, sorted=1;
4     scanf("%d", &n);
5     for(i=0; i<n; i++)
6         scanf("%d", &a[i]);
7     scanf("%d", &key);
8     for(i=0; i<n-1; i++)
9         if(a[i]>a[i+1])
10            sorted =0;
11
12     if(sorted)
13         printf("binary search");
14     else
15         printf("linear search");
16     return 0;
17 }
```

Run

Save

Input

5
10 20 30 40 50
30

Output

```
26     }
27     if (isSym)
28         printf("Symmetric matrix\n");
29     if (isUpper)
30         printf("Upper triangular matrix\n");
31     if (isLower)
32         printf("Lower triangular matrix\n");
33     if (isDiag)
34         printf("Diagonal matrix\n");
35     if (isId)
36         printf("Identity matrix\n");
37     return 0;
38 }
```

 Run

 Save

CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks

Q2
CO3 - AT1 - Analytical Problem Solving
20 marks

Q3
CO3 - AT1 - Analytical Problem Solving
20 marks

Q4
CO3 - AT1 - Analytical Problem Solving
20 marks

Q5
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q1 CO3 - AT1 - Analytical Problem Solving

20 marks

1. Given an $N \times N$ matrix, Write a C Program to determine whether it is:

- Symmetric
- Upper triangular
- Lower triangular
- Diagonal matrix
- Identity matrix
- Display all applicable classifications.

Your Code

```
1 #include <stdio.h>
2 int main() {
3     int n;
4     scanf("%d", &n);
5     int a[50][50];
6     int i, j;
7     for (i = 0; i < n; i++) {
8         for (j = 0; j < n; j++) {
9             scanf("%d", &a[i][j]);
10        }
11    }
12    int isSym = 1, isUpper = 1, isLower = 1, isDiag = 1, isId = 1;
13    for (i = 0; i < n; i++) {
14        for (j = 0; j < n; j++) {
15            if (a[i][j] != a[j][i])
16                isSym = 0;
17            if (i > j && a[i][j] != 0)
18                isUpper = 0;
19            if (i < j && a[i][j] != 0)
20                isLower = 0;
21            if (i != j && a[i][j] != 0)
22                isDiag = 0;
23            if ((i == j && a[i][j] != 1) || (i != j && a[i][j] != 0))
24                isId = 0;
25        }
26    }
27    if (isSym)
28        printf("Symmetric matrix\n");
```

Input

3
1 0 0
0 1 0
0 0 1

Output

Act
Go t